

Lab Course Report: "Make Car Go Fast"

Simon Wilhelm Schübel
Karlsruhe Institute of Technology
Karlsruhe, Germany
Email: uerga@student.kit.edu

Tahmur Hassan Mahmood
Karlsruhe Institute of Technology
Karlsruhe, Germany
Email: uxgzc@student.kit.edu

Abstract—This student project focuses on training a reinforcement learning (RL) agent for the game Trackmania Nations Forever (TMNF). The policies for the agents we investigate are trained using different kinds of observations, rewards, and maps. We created a Gymnasium-compatible framework to be able to train reinforcement learning agents, which we built using existing trackmania-specific tools. Using this framework, we are able to train RL agents using existing implementations of RL algorithms. In contrast to existing reinforcement learning projects on TMNF, like e.g., Linesight, our framework is compatible with existing RL libraries, accelerating development and testing. Our agents were able to master the maps we trained them on and even achieved human-comparable performance.

I. INTRODUCTION

Reinforcement Learning (RL) is a powerful framework for learning decision-making policies in environments that require sequential reasoning. In particular, visual RL has demonstrated impressive results, achieving superhuman performance in games such as ATARI and racing simulations like Gran Turismo and Trackmania Nations Forever (TMNF) [22, 18]. These games are appealing benchmarks because they combine high-dimensional visual input with complex control dynamics, making them ideal for testing agents in realistic, interactive settings. Especially for autonomous systems, such as self-driving cars, the ability to learn from raw visual observations without relying on handcrafted rules is a significant advantage [10, 11, 25].

This work introduces a flexible and extensible gymnasium environment [32] designed to define and train agents across a variety of Markov Decision Processes (MDPs). Unlike existing frameworks, our environment focuses on modularity and generalization. It enables researchers to explore a wide range of configurations, without being constrained by hard-coded logic or inflexible design.

Using our framework, we evaluated various reward functions in both sparse and dense settings, ultimately focusing on dense rewards due to promising early results. Through curriculum learning, agents refined their behavior and achieved human-level performance.

II. FUNDAMENTALS

A. Trackmania Nations Forever

Trackmania Nations Forever is a free-to-play racing game developed by Nadeo and released in 2008. Players control a car on highly varied maps, ranging from simple stadium circuits to complex layouts with sharp turns, jumps, loops,

and wall-rides. The main mode of TMNF used in this study is *Time Attack*, where the objective is to complete a lap in the shortest time possible. Achieving this demands pixel-perfect driving, optimal pathfinding, and precise control of the car. These challenges make TMNF an interesting environment for RL research [17, 20].

We use TMNF as a simulator. Third-party tools, like TMInterface, give access to internal game-states and frames. Instead of treating TMNF as a continuous game, we interact with it as a simulator, which is discretely steppable.

B. TMInterface

TMInterface (TMI) [9] is a third-party tool developed by Donadigo. TMI is the de facto standard *Tool Assistant Speedrun* (TAS) tool for TrackMania Nations Forever. On the one hand, TMI is an important modification to the players of TMNF that use it to enhance their performance in-game. On the other hand, and more importantly for this project, it provides a Python client allowing third-party software, like ours, to directly interact with the game, without human intervention.

The Gamespeed of TMNF controls the in-game time passed between two simulation steps. The game speed at which TMNF is running, if humans play it, is usually 1. Using TMI, it's possible to increase the game speed, which is beneficial for training, as it increases the trajectory-collection speed.

C. Linesight

Linesight Trackmania AI [1], or simply *Linesight*, is one of the most popular reinforcement learning projects for TMNF. It attracted and built a thriving community of developers. It had huge public exposure, bringing the concept of reinforcement learning within TMNF to millions of people on YouTube ¹. It is especially interesting for us as it sets a baseline standard for RL-agents. Furthermore, although the code quality and expandability of the linesight-repository is not up to any industry standard, it was and still is a standard entry point for developers who aim to conduct research or private projects using TMNF.

D. Reinforcement Learning

Reinforcement Learning is a branch of machine learning in which a software agent learns a policy to act in an

¹"I Trained an AI for 2 Years on Trackmania. It's Breaking Records." (Accessed 12.07.2025).

environment in a way that maximizes its returns by interacting with the environment. Formally, the environment is modeled as a Markov Decision Process. A MDP can be represented as a tuple: (S, A, R, ρ, s_0) , which consists of a set of states S , sometimes also called observations, a set of actions A , a reward function $R : S \times A \rightarrow \mathbb{R}$ as well as a, possibly stochastic, state transition function $\rho : S \times A \rightarrow S$ and a starting state $s_0 \in S$ [14]. Each interaction of the agent with its environment happens at a discrete point in time $t \in \mathbb{N}_0$; for each timestep t , the agent is in state s_t and executes action $a_t = \pi(s_t)$. For this action, the reward $r_t = R(s_t, a_t)$ is calculated. Afterwards the agent is in state $s_{t+1} = \rho(s_t, a_t)$ and the cycle continues. The mathematical goal of the agent in each time step is to maximize its expected discounted sum of rewards, also called return

$$J_t(\pi_\theta) = \mathbb{E}_\pi[G_t], \quad G_t = \sum_{k=0}^T \gamma^k \cdot r_{t+k}.$$

The discount factor $\gamma \in [0, 1]$ acts as a weight for future rewards and balances how important immediate rewards are in relation to future rewards. Usually, $\gamma = 0.99$ provides a good balance, as it ensures convergence of G_t , yet also emphasizes long-term rewards [31]. The agent's goal is to find a policy $\pi : S \rightarrow A$, which maximizes its return. In Deep Reinforcement Learning, this policy is represented by neural networks, which in our end-to-end case, directly take in the observations as inputs and output a policy $\pi(s)$ [19].

1) *Approaches and Algorithms*: There exist various methods to train an agent that maximizes its return. Very generally, these methods can be classified into Model-Free RL and Model-Based RL [31]. In contrast to Model-Free RL, in Model-Based RL methods, the agent is either given the model or tries to learn the model of its environment [16]. Popular instances of Model-Free RL methods include Value-Based approaches and Policy-Based approaches [28]. Specific algorithms can be classified into on-policy and off-policy algorithms. On-policy algorithms use the same policy to generate actions and collect data, while off-policy algorithms use a behavior policy to collect data in order to optimize a target policy. [12, 6].

a) *Value-Based approaches*: Value-Based approaches focus on estimating a state-value function $V(s)$ and an action-value function $Q(s, a)$, which are used to derive a policy for the agent. The state-value function describes the cumulative reward when behaving according to a given policy π , while the action-value function describes the cumulative future reward when taking action a in state s and behaving according to policy π afterward, i.e.,

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t \cdot r_t | s_0 = s \right],$$

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t \cdot r_t | s_0 = s, a_0 = a \right].$$

Q-learning [35] is a popular RL algorithm that works by learning an optimal action-value function $Q^*(s, a)$ using the

Bellman optimality Equation [2] and updates the Q-values iteratively. The Bellman optimality Equation for the action-value function is defined as :

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_a Q^*(s_{t+1}, a_{t+1}) | s_t, a_t \right] \quad (1)$$

The policy is then derived as $\pi(s) = \arg \max_a Q(s, a)$. Value-based methods can be very efficient for small, discrete states and action spaces and are straightforward to implement [35]. Deep Q-Network (DQN) is a popular DRL algorithm that represents the Q-function using a neural network [21]. Quantile Regression DQN (QR-DQN) [7] improves on DQN by learning a distribution over returns using quantile regression, allowing the agent to capture return variability more effectively. Instead of a single Q-value, QR-DQN estimates a set of quantiles $\{\theta_i(s, a)\}_{i=1}^N$, representing the return distribution

$$Q(s, a) \approx \frac{1}{N} \sum_{i=1}^N \theta_i(s, a).$$

b) *Policy-Based approaches*: Policy-Based approaches directly optimize the policy. The policy in this case is a probability distribution over actions given a state, defined by parameters θ : $\pi_\theta(a|s)$ describes the probability of taking action a in state s . A prominent member of policy-based approaches is Policy Gradient methods, which use the gradient of the objective function $J_t(\pi_\theta)$ with respect to the policy parameters θ to optimize the policy [36, 27]. The policy gradient used to optimize π using gradient ascent can be defined as

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi \left[\nabla_\theta \log \pi_\theta(a|s) \cdot G_t \right].$$

c) *Actor-Critic algorithms*: Actor-Critic algorithms depict a popular class of RL algorithms, as they are a combination of both policy-based and value-based approaches [23]. Actor-Critic algorithms work by learning a policy π , called the actor, and a value function, called the critic. Similarly to policy-based approaches, actor-critic algorithms use a policy, which means $\pi_\theta(a|s)$ describes the probability of taking action a in state s with parameters θ . The actor chooses actions for a given state based on the current policy. The task of the critic is to evaluate the actions of the agent by estimating the value of the state. This estimation is used to calculate a temporal difference error $\delta = r_t + \gamma V(s_{t+1}) - V(s_t)$. The critic as well as the actor updates the values for their functions based on this error, i.e., the loss of a prominent actor-critic algorithm, Proximal Policy Optimization (PPO) [27], consists of

$$L_t^{CLIP+VF+S}(\theta) = \mathbb{E}_t[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)]$$

where L^{CLIP} is the clipped surrogate objective and L^{VF} is the squared temporal difference error, while S is an entropy bonus and c_1, c_2 are coefficients.

E. Algorithm Choice

Model-based algorithms offer superior sample efficiency by leveraging learned environment models, but remain challenging to train due to compounding model errors and instability in

long-term predictions [5, 34]. While Model-based reinforcement learning is effective when the environment interaction is costly (e.g., robotics) [8], TMNF is a fast, resettable simulation environment, which means data collection is relatively inexpensive. Given this environment setting, model-free methods, despite their sample inefficiency, become a practical choice due to their simplicity and strong empirical performance [13]. Therefore, we selected PPO for its stability in on-policy learning [27] and QR-DQN for its improved return distribution modeling over standard DQN [7].

III. GENERAL SETUP

In this section, our goal is to introduce the code framework we used to train our agents, as well as the problems we encountered and solved during development.

A. Unified Framework Approach

The incompatibility of existing RL-frameworks for TMNF, like *Linesight*, with other, industry-standard RL-libraries like Gymnasium [32] or Stable-Baselines3 [24], poses a challenge. We faced the same problem and therefore created a Gymnasium-Conform modular reinforcement learning environment, which allowed us to use all existing implementations of gymnasium-compatible RL-algorithms, and also will hopefully pave the way for future RL-researchers to focus on the RL part.

1) *Communication*: The main idea behind the framework is to encapsulate the communication between TMNF-Game and TMI into its own process, as illustrated in Figure 1. The environment then sends a step-like command, which consists of an action, and the TMInteraction Process returns corresponding observations.

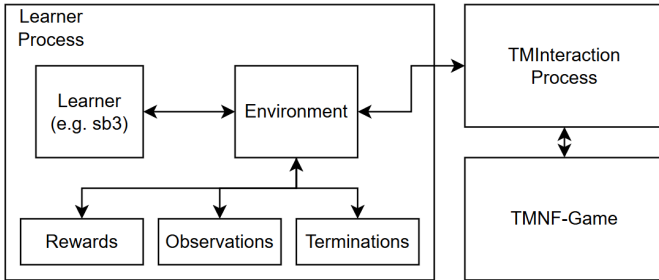


Fig. 1. Overview of the communication framework. The environment and the learner are in one process, communicating with the TMInteraction Process using inter-process communication protocols. The TMInteraction Process is the only one interacting with the game directly.

2) *Synchronization Issue*: One of the main issues we faced was the synchronization of the simulator and the environment. We observed that without proper synchronization, the number of simulation steps between each environment step is not consistent with the game speed. This is illustrated in Figure 2. The first major complication this causes is that an agent trained on gamespeeds a certain cannot transfer its policy to perform inference at any other gamespeed. The second problem, which

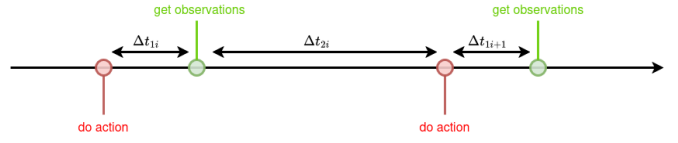


Fig. 2. Time Intervals between action, observation, and next environment step. Δt_{2i} is the main desynchronization gap.

is closely related to the first, is an issue with training itself. Due to the inconsistent amount of simulation steps between environment steps, the simulator has simulated many too many steps between two actions of the agent, creating a gap of where the agent thinks the car is, to where the car is actually at, as can be seen in Figure 3.



Fig. 3. Unsynchronized Image-Observations exactly one environment step apart at gamespeed 50. In the first image (left), a turn in the track is clearly visible; in the next environment step (right), the car has skipped the entire length of the straight and crashed directly into the wall.

3) *Race Finished Animation*: If a player in TMNF finishes a game, the game automatically displays a race-finished animation. During training, this animation breaks the link between TMNF and the TMInteraction Process. Although TMI provides functionality that enables one to skip this animation and go straight to the finish line, it is unreliable. We were not able to eliminate this issue, yet we worked around it by restarting the environment once this happened. This ensures a continuous learning process, yet the trajectory at which this restart happens is not consistent, as the reward where the agent crosses the finish line is that of the first step of a resetted environment. We attribute this miscalculation to environmental noise.

IV. TRACKS

The tracks define a substantial part of the environment itself, as they dictate the course the agent has to drive on and master. In general, we used two different tracks. At the beginning of the development, we had problems using pre-existing tracks. We therefore created this *Simple-Validated* (Track A). It features serpentine-like curves in the beginning and is also built on three different levels to introduce variety and difficulty. *Curvy* (Track B) is a much simpler track than *Simple-Validated*, consisting of many ground-level curves,

resulting in a serpentine-like track from start to finish. This track is also heavily used in the RL community for TMNF. In-game images of both tracks are displayed in figure 4.

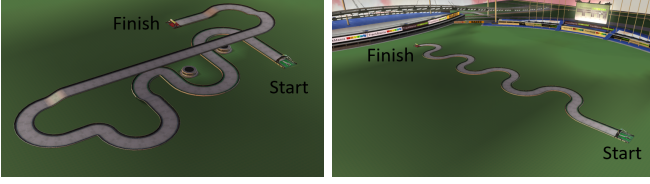


Fig. 4. In-game image of Track A (left) and Track B (right). This screenshot was taken with a different light setting than was used for training.

A. Reference Line

The reference line $L \subseteq \mathbb{R}^{N \times 3}$ consists of N points, defined in the global track-coordinate system. In order to create these points, we had to drive each track manually, after which we can extract approximately equidistant points along the driven trajectory. The main reason for the reference line is as a means of measuring progress along the track.

The progress of the agent along the track is defined by the reference line. At every timestep t , the agent has a next reference-line point $p_t \in L$, which it has not yet passed. In each step, the position of the agent is compared to the reference line, and that point, which the agent has not yet passed and is closest to, becomes the new next reference line point p_t .

V. MDP FORMULATION

In this section, we want to introduce the MDP (S, A, R, ρ, s_0) that models the reinforcement learning environment. In particular, this section introduces the actions A , Observations S , and reward function $r_t = R(s_t, a_t)$.

1) *Environment Steps vs. Simulation Steps*: Simulation steps are the steps performed by the simulator, in our case, the game TMNF. To achieve invariance against different game-speeds, or simulation-speeds, we fixed the number of actions per second to 20. This subsequently defines the number of simulation steps for each environment step.

A. Actions

The actions of the agent are discrete and the same for all experiments. Although discrete actions are not desirable for steering a car, they reflect a similar input as available to humans, which consists of only the four inputs "gas", "brake", "left", and "right" in each environment step. Combining these four into every possible combination gives us 12 discrete actions. Each action is represented as $a \in \{0, 1\}^4$, a four-dimensional Boolean vector, which is 1 if the corresponding action is chosen. The action reduction from 16 to 12 actions is achieved by ignoring canceling actions, e.g., ("left" = true, "right" = true).

B. Observations

All of our tested observations consist of a state-vector $s \in \mathbb{R}^d$, containing vector-like information and an image $\in \mathbb{R}^{w \times h}$. We did not add noise to any of our observations, as in our case, the simulator (TMNF) is also the real-world application of the agent.

1) *Image-Observations*: Our image observation is a 128×128 grayscale image, as displayed in Figure 5.



Fig. 5. Example 128×128 grayscale example observation during training. We removed all graphical display elements, e.g., the time-driven or the speed.

2) *Vector-Like Observations*: The state vector contains information directly extracted from the game, as well as the next $k(= 10)$ reference-line points in the car coordinate system. A complete list of observations along with their dimensionality is provided in Table I.

3) *Normalization*: In order to normalize the images and state vector, we divided the image element-wise by 255 to obtain a normalized image. For the state vector, we defined maximum values for each type in the observation vector and divided the observation given by the observation manager by that value.

C. Rewards

We designed the reward function of the agent as a weighted combination of multiple components

$$r_t = r_t^a + \lambda^l r_t^l + \lambda^v r_t^v + \lambda^f r_t^f + \lambda^g r_t^g + \lambda^s r_t^s + \lambda^p r_t^p. \quad (2)$$

1) *Accumulated Distance*: The accumulated distance reward measures the progress of the car along the center line by adding the distance of all points passed together, normalized by the total distance

$$r_t^a = \sum_{k=i_{t-1}}^{i_t} \frac{L[0 : k] - L[0 : k - 1]}{L[0 : N]}.$$

2) *Lateral Distance*: The lateral distance reward measures the distance of the position of the agent to the reference line. Although the reference line was driven manually, it is roughly in the center of the track; therefore, the lateral distance is a measure of how far the agent is from the center.

$$d_{lateral} = \sqrt{\|d\|^2 + \left(\frac{d^T s}{\|s\|}\right)^2}$$

where $s = p_t - p_{t-1}$ is the vector of the current reference-line point to the last reference line point and $d = p_{car} - p_{t-1}$ is the vector of the position of the car to the last reference line point. $d_{lateral}$ is illustrated in Figure 6. In order to achieve

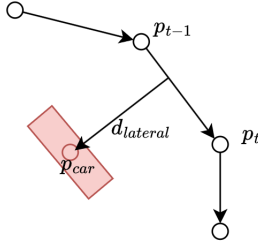


Fig. 6. Illustration of lateral distance, where arrows are the links between reference line points. The car (red rectangle) is not to scale.

a smoother interpolation of the reward over the width of the track, we scale this reward by first normalizing it and then evaluating its probability over a scaled and shifted Gaussian, as shown in Figure 13.

$$r_t^l = m \cdot \mathcal{N}\left(\frac{d_{lateral}}{\gamma} \middle| 0, 1\right) + y \in [1, -1] \quad (3)$$

This way, we control how much of the track around the reline gets a positive reward and what part gets a negative.

3) *Constant Rewards and Punishments*: Constant rewards are fixed values given to the agent either at every environment step or during specific events. These events include completing the race r_t^g , getting stuck (being unable to move) r_t^s , or failing to make progress along the center-line r_t^p . Because our task equates to a so-called minimum-time task, we also give a small constant negative reward r_t^f at each step before the race is completed. Although seeming uninformative, constant negative rewards can help improve the policy [33].

4) *Normalization*: In order to normalize the rewards, we chose to use a running average that tracks both a running mean and variance:

$$r_{normalized} = \frac{r_t - \mu}{\sqrt{\sigma + \epsilon}}.$$

To track batch-wise statistics (μ and σ^2), we used Stable-Baselines3 [24] implementation of a running mean.

D. Termination

The Episode ends after a fixed timeout of 4096-environment-steps, or one of the other termination conditions, which are: the car is stuck, the agent finished the race, or (but rarely used), the agent did not make any progress along the reference line. In all of these scenarios mentioned above, except for a regular timeout, the agent is dealt a reward.

1) *No Progress*: The main motivation behind implementing the no-progress condition was to prevent the agent from persisting in behaviors such as prolonged off-track driving or moving in reverse, which yield uninformative trajectories and therefore result in inefficient use of training time. However, subsequent analysis revealed that this mechanism was ultimately unnecessary.

VI. NETWORK

The network representing the agent consists of a feature extractor, which contains a vision model, as well as a MDP-feature extractor for vector-like observations. The feature extractor defines an output layer, after which the latent representation of the observations is given to the policy and critic network, as can be seen in Figure 7.

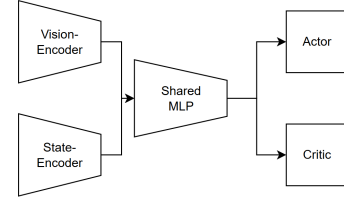


Fig. 7. Shared Feature-Encoder Pipeline. The latent representation of Vision-Encoder and State-Encoder are concatenated, resulting in a 128-dimensional vector, which is then given to the shared-MLP, which creates a latent representation of the whole observation vector.

A. Vision Encoder

The vision encoder, described in IV, had to be small due to the hardware-based limitations of the PC we trained on. We therefore decided on a traditional Convolutional Neural Network, which has been proven to work in vision-based reinforcement learning projects [15].

VII. EXPERIMENTS

This section introduces the conducted experiments and their outcomes. Rather than comparing experiments based on absolute reward values, which are difficult to interpret due to differences in scaling, normalization, and track configurations across different experiments, we use mean episode length as the primary performance metric. This also corresponds to the goal to maximize in *Time Attack*. While we did conduct many more experiments than listed, the listed ones resemble our main breakthroughs and insights. All introduced experiments use 128×128 grayscale images and the vector-like observations as stated in I.

A. First successful agent

Using the rewards given in Table II, we trained the first promising agent. We trained the agent on Track A, and for the first time, the agent learned to reach the finish line consistently, i.e., 40% of all runs. We trained this agent for around 12 million environment steps, but observed stagnation in the mean episode length after around four million steps, as can be seen in Figure 8. It only shows eight million environment steps, which is due to a restart. At this point, we had not yet figured out how to counter the race-finish animation bug, which is why we lost the first 4 million steps. The very prominent drop in episode length at around 7 million steps is not due to the agent driving extremely fast, but rather due to the agent consistently getting stuck at around two-thirds of the track. We can't explain why this happened or why this lasted for almost 200,000 steps. Nevertheless, the final and average agent managed to complete the race consistently with a mean episode length of 1000 environment steps. This performance would be considered below-average human performance.

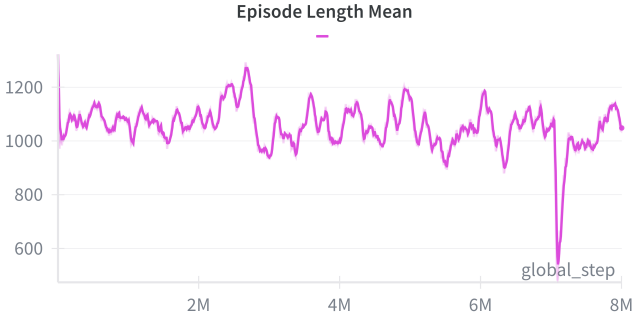


Fig. 8. Mean episode length during rollout of the agent using the dense-reward function, trained with PPO.

1) *Wall-Bumping*: The main reason for the agent's poor performance was that the agent repeatedly bumped into the track walls. Not only on sharp corners, but also on straights. We can't explain this behaviour, especially because the lateral distance was weighted almost a third of the whole reward signal, as can be seen in Table II.

2) *Lack of Normalization*: It is important to note that for this experiment, only the images were normalized to the range of $[0, 1]$. Although the reward-weights were chosen, such that the final reward signal would roughly be within $[0, 1]$, these were only estimated scaling of the weights and often did not remain within $[0, 1]$. More importantly, the observation vector was not normalized.

B. Comparing off-policy to on-policy algorithms

After training the first successful agent, we wanted to test whether an off-policy algorithm yielded better training efficiency than the implementation of PPO we were using before. We selected QR-DQN as the off-policy algorithm. To test the performance, we used the exact same MDP configuration as

for the PPO agent to train the QR-DQN agent. As can be seen in Figure 9, QR-DQN achieves similar agent performance in terms of the finished races and race time measured by the mean episode length as PPO after only one million steps. It is furthermore important to note here that the PPO agent was already training for 4 million steps, while the QR-DQN run was starting from scratch. In terms of reward-performance, the QR-DQN run outperformed PPO by some margin, as can be seen in Figure 14.

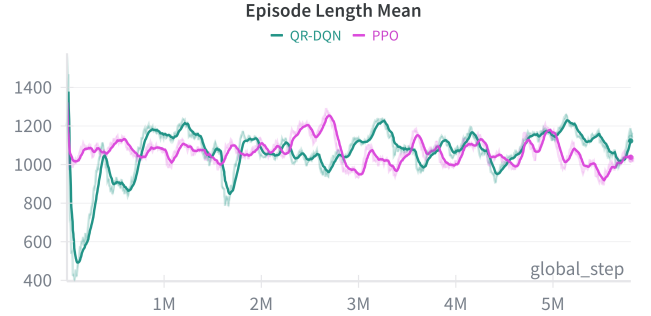


Fig. 9. Episode mean length of QR-DQN (green) compared to PPO (pink).

1) *Drawbacks*: Although the QR-DQN run quickly caught up to the PPO-run in terms of agent-performance and even marginally outperformed the reward of the PPO agent, it still suffered from Wall-Bumping and lack of normalization. Furthermore, the computational performance of QR-DQN was significantly worse than PPO, taking almost 4 times the time to complete similar amounts of environment steps, and still not running with desirable hyperparameters. The updates of the QR-DQN run were unstable, especially striking in 14, which we suspected was due to the low batch size we were using. Due to hardware constraints, the batch size could not be larger than 64. We trained on a batch size of 32, because it was almost twice as fast, and also, the increase to 64 made no difference, as can be seen in Figure 15. It was for these reasons that we continued experimenting with PPO.

C. Sparse Reward setting

Following prominent reinforcement learning experiments that used sparse rewards [29, 26], we also wanted to try a sparse reward setting instead of a very dense-informative reward, as described in Equation (2). We reduced the reward signals only to the accumulated distance and a dynamically scaled race-finished reward

$$r_t^g = 1 - \frac{s}{n} \in [0, 1],$$

where s is the number of environment steps until the race was finished, and n is the number of environment steps until timeout. However, this reward modification did not work, as on Track A, the agent could not even climb the first ramp, due to insufficient speed. Even after millions of environment steps, the agent was not able to overcome the ramp once.

Following this, we tried this reward setting on *Curvy*. We even created a shorter version of Track B in order to ensure that the agent, even if it may be by accident, reached the goal more frequently, which would help the policy to converge faster. In all of these configurations of Track B, the agent was not able to reach the finish line consistently, getting stuck and not achieving to learn anything. We attribute this to the poor performance of the critic. As shown in Figure 10, none of the configurations yielded substantial explained variance, with the highest value observed in Scenario 1 at only 0.69. Despite this peak, the explained variance curves for all three scenarios remain asymptotically flat, aside from a few extreme negative outliers. Similarly, other performance metrics, such as the mean episode length and mean episode reward (see Figure 16 for reference), exhibit the same asymptotically flat behavior. In fact, all three agents failed to consistently finish the race.

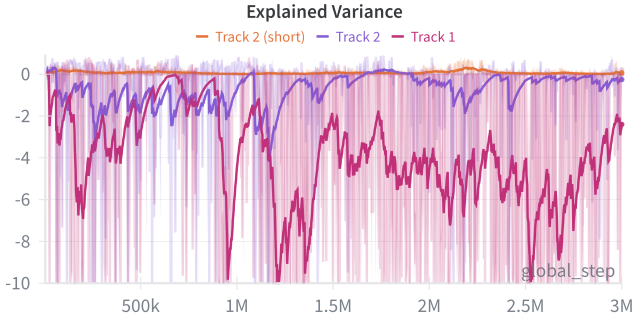


Fig. 10. Explained Variance for Track A (red), Track B (purple), and Track B shortened (orange).

D. Normalization and Learning Rate-Decay

This experiment uses the same MDP as our first experiment, but with two major improvements. The first is the introduction of a learning rate scheduler. The second improvement is the normalization of both rewards and all observations. We normalized every entry in the state vector. The results of this change are not to be understated, as the trained agent significantly outperforms the agent from our original setup, being almost twice as fast as can be seen in Figure 11. While the fastest race time for the unnormalized PPO agent was around 900 environment steps per race, the fastest race time for the normalized PPO agent was 714 environment steps.

1) *Learning Rate Schedule*: The learning rate schedule we used consists of a constant schedule until half of the training steps are done, and then resorts to a linear schedule

$$\text{lr}_\beta(\varrho) = \begin{cases} 2.5 \cdot 10^{-4} & \varrho < \beta \\ 2.5 \cdot 10^{-4} \cdot \varrho & \varrho \geq \beta \end{cases}$$

where $\varrho \in [0, 1]$ is the progress of the training (0 at the beginning, 1 at the end) and $\beta \in [0, 1]$ is the amount of progress after which the scheduling shall begin. In our case, we chose $d = 0.5$. We chose this due to the oscillating behaviour

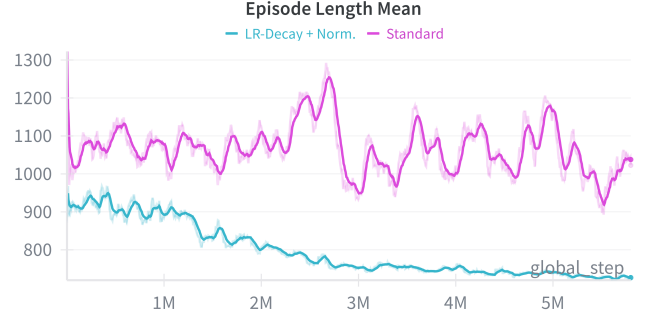


Fig. 11. Comparison of mean episode length of normalized PPO (blue) to regular PPO (pink).

of the runs after an initial training phase. Interestingly, even for the first half of the training, in which both normalized and unnormalized have the exact same learning rate, the normalized agent significantly outperforms the unnormalized.

2) *Extreme Midlane Cruising*: The normalized PPO agent maximized his reward function so well that he completely eliminated wall-bumping, but drove in the middle of the road at all times. Although it is impressive to see how direct and well the impact of the lateral-distance reward works on the agent's behaviour, it is suboptimal in terms of finding the fastest time from start to finish, because the agent does not find optimal lines.

E. Removing Lateral Distance

Due to this strict adherence to the reference line, we wanted to test what happens if we just remove the lateral distance reward. Because the agent only needs to learn to find optimal lines in corners, we decided to resume training from the previous agent after 5 million environment steps in a curriculum learning fashion, which has been proven to be successful in reinforcement learning for racing games [30]. The results were astonishing, as the agent further increased his performance by learning to drive optimal lines in corners. The mean length per episode during training can be seen in Figure 12. The agent managed to further decrease the time taken to finish the race to 629 environment steps, which, on average, is a 33% improvement to our initial Agent trained with this reward function, which is a competitive, but not record-breaking, race time for humans.

F. Human Benchmark

In order to give insight into how the agents compare to humans playing the game, we asked 10 people to play Track A and Track B. To get a comparable measurement from each person, we allowed them a training phase, where they could each play the Track three times before the fourth time, when we took the measurement. While the full evaluation of human performances can be found in Table III, the best times achieved for Track A were 621 environment steps (or 31.05 seconds) and for Track B 422 environment steps (or 21.1 seconds). Each

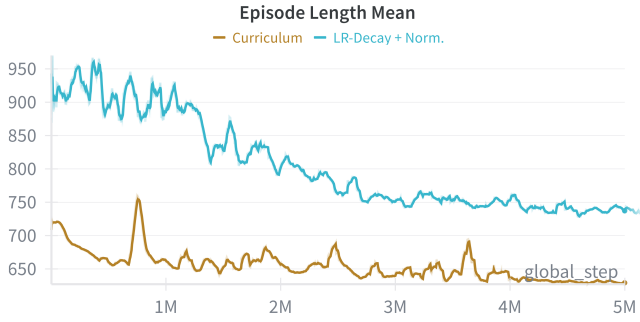


Fig. 12. Comparison of mean episode length of agents trained using PPO and reward function (2) (blue) with an agent that is trained in the same context and not given the lateral distance reward.

participant took part willingly and was not compensated. The skill level varied among the Volunteers, where some played video games, including TMNF, regularly, while others didn't play video games at all.

G. Generalization

In order to test the generalization capabilities of our agents, we conducted performance tests on unseen tracks. Because all of the agents were originally trained on Track A, we let them drive on Track B. Surprisingly, the agents also managed to reach the goal consistently on this track. However, their performance was considerably worse, dropping from being competitive compared to a human to being laughed at. To test the limits of our trained agents, we also tested them on the first in-game level, called *A01-Race*. The agents, however, could not overcome a crucial jump in the track. This is expected, as this jump is completely out of distribution for the agents. We believe the similarity of Track A and Track B in terms of height variations and necessary jumps is what makes it possible for the agent to transfer some knowledge.

VIII. CONCLUSION

This project successfully developed a modular and extensible reinforcement learning framework for training agents in Trackmania Nations Forever, a complex racing environment that requires high-precision control and visual understanding. Through a series of carefully structured experiments, we demonstrated that our Gymnasium-compatible interface enables integration with industry-standard RL algorithms and libraries, substantially improving development flexibility compared to prior work. The agents, trained using PPO and QR-DQN, achieved human-comparable performance on custom-designed tracks. We found that dense reward structures, particularly those combining accumulated and lateral distance metrics, outperformed sparse reward settings. A major insight was that normalization of both observations and rewards, along with curriculum learning strategies, significantly improved training stability and performance. In the end, we were able to reduce race completion times by over 30% compared

to our initial experiments. However, our agents displayed limited generalization to unseen tracks, struggled with optimal pathfinding (e.g., wall-bumping or overly conservative driving), and failed to surpass skilled human performance. Moreover, off-policy methods like QR-DQN, while sample-efficient, presented practical drawbacks in terms of training speed and stability on limited hardware.

A. Future Work

The next steps for continuing this work include the implementation of a headless Trackmania version, as also proposed by other projects, such as [17]. This is necessary to leverage more powerful hardware and train on computer clusters. Furthermore, a vectorized version of the environment could be used to parallelize trajectory collection, which would reduce training time and allow simultaneous training on multiple tracks.

Beyond implementation-level improvements, a promising direction for future research is the use of foundation models. Foundation models are large-scale, pre-trained models that can be fine-tuned for a wide range of downstream tasks with minimal task-specific data [3]. In the context of reinforcement learning, training a foundation model as a general-purpose agent could improve generalization across different track layouts, driving styles, and environmental variations.

Another direction worth exploring is the integration of Model Predictive Control, which has shown strong performance in racing and control tasks [4]. Model Predictive Control enables the agent to plan by simulating several future trajectories based on a dynamics model and selecting the optimal sequence of actions. This approach can improve short-term decision-making and safety in high-speed environments like racing.

ACKNOWLEDGMENT

The authors would like to thank donadigo for providing TMInterface and its documentation; without it, this project would not have been possible in the capacity that it has been.

REFERENCES

- [1] Agade09 (GitHub-Username) and pb4git (GitHub-Username). *Linesight-AI Documentation*. <https://linesight-rl.github.io/linesight/build/html/>. Accessed: 17 August 2025. 2025.
- [2] Richard Bellman. "Dynamic programming and stochastic control processes". In: *Information and Control* 1.3 (1958), pp. 228–239. ISSN: 0019-9958. DOI: [https://doi.org/10.1016/S0019-9958\(58\)80003-0](https://doi.org/10.1016/S0019-9958(58)80003-0). URL: <https://www.sciencedirect.com/science/article/pii/S0019995858800030>.
- [3] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, et al. "On the Opportunities and Risks of Foundation Models". In: *arXiv preprint arXiv:2108.07258* (2021).
- [4] Francesco Borrelli, Alberto Bemporad, and Manfred Morari. *Predictive Control for Linear and Hybrid Systems*. Cambridge University Press, 2017.

- [5] Kurtland Chua et al. *Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models*. 2018. arXiv: 1805.12114 [cs.LG]. URL: <https://arxiv.org/abs/1805.12114>.
- [6] Nicholas E. Corrado and Josiah P. Hanna. *On-Policy Policy Gradient Reinforcement Learning Without On-Policy Sampling*. 2024. arXiv: 2311.08290 [cs.LG]. URL: <https://arxiv.org/abs/2311.08290>.
- [7] Will Dabney et al. *Distributional Reinforcement Learning with Quantile Regression*. 2017. arXiv: 1710.10044 [cs.AI]. URL: <https://arxiv.org/abs/1710.10044>.
- [8] Marc Peter Deisenroth, Dieter Fox, and Carl Edward Rasmussen. "Gaussian Processes for Data-Efficient Learning in Robotics and Control". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 37.2 (2015), pp. 408–423. DOI: 10.1109/TPAMI.2013.218.
- [9] donadigo#6525. *tminterface*. <https://donadigo.com/tminterface/>. Accessed: 17 August 2025. n.d.
- [10] Andreas Folkers, Matthias Rick, and Christof Buskens. "Controlling an Autonomous Vehicle with Deep Reinforcement Learning". In: *2019 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, June 2019, pp. 2025–2031. DOI: 10.1109/ivs.2019.8814124. URL: <http://dx.doi.org/10.1109/IVS.2019.8814124>.
- [11] Florian Fuchs et al. "Super-Human Performance in Gran Turismo Sport Using Deep Reinforcement Learning". In: *IEEE Robotics and Automation Letters* 6.3 (July 2021), pp. 4257–4264. ISSN: 2377-3774. DOI: 10.1109/lra.2021.3064284. URL: <http://dx.doi.org/10.1109/LRA.2021.3064284>.
- [12] Shixiang Gu et al. *Q-Prop: Sample-Efficient Policy Gradient with An Off-Policy Critic*. 2017. arXiv: 1611.02247 [cs.LG]. URL: <https://arxiv.org/abs/1611.02247>.
- [13] Peter Henderson et al. *Deep Reinforcement Learning that Matters*. 2019. arXiv: 1709.06560 [cs.LG]. URL: <https://arxiv.org/abs/1709.06560>.
- [14] Ronald A. Howard. *Dynamic Programming and Markov Processes*. New York: John Wiley & Sons, 1960.
- [15] Shirin Joshi, Sulabh Kumra, and Ferat Sahin. *Robotic Grasping using Deep Reinforcement Learning*. 2020. arXiv: 2007.04499 [cs.RO]. URL: <https://arxiv.org/abs/2007.04499>.
- [16] Lukasz Kaiser et al. *Model-Based Reinforcement Learning for Atari*. 2024. arXiv: 1903.00374 [cs.LG]. URL: <https://arxiv.org/abs/1903.00374>.
- [17] Pius Kriemler. *Distributed Setup for Trackmania Reinforcement Learning*. <https://pub.tik.ee.ethz.ch/students/2023-FS/GA-2023-03.pdf>. Bachelor's Thesis, ETH Zurich. 2023.
- [18] Hojoon Lee et al. "A Champion-Level Vision-Based Reinforcement Learning Agent for Competitive Racing in Gran Turismo 7". In: *IEEE Robotics and Automation Letters* 10.6 (2025), pp. 5545–5552. DOI: 10.1109/LRA.2025.3560873.
- [19] Guisong Liu et al. "Human-Level Control Through Directly Trained Deep Spiking Q-Networks". In: *IEEE Transactions on Cybernetics* 53.11 (Nov. 2023), pp. 7187–7198. ISSN: 2168-2275. DOI: 10.1109/TCYB.2022.3198259. URL: <http://dx.doi.org/10.1109/TCYB.2022.3198259>.
- [20] Girolamo Macaluso, Alessandro Sestini, and Andrew D. Bagdanov. *A Benchmark Environment for Offline Reinforcement Learning in Racing Games*. 2024. arXiv: 2407.09415 [cs.AI]. URL: <https://arxiv.org/abs/2407.09415>.
- [21] Volodymyr Mnih et al. "Human-level control through deep reinforcement learning". In: *Nature* 518.7540 (2015), pp. 529–533. DOI: 10.1038/nature14236.
- [22] L.J. Neinders. *Improving Trackmania Reinforcement Learning Performance : A Comparison of Sophy and Trackmania AI*. May 2023. URL: <http://essay.utwente.nl/96153/>.
- [23] Deirdre Quillen et al. *Deep Reinforcement Learning for Vision-Based Robotic Grasping: A Simulated Comparative Evaluation of Off-Policy Methods*. 2018. arXiv: 1802.10264 [cs.RO]. URL: <https://arxiv.org/abs/1802.10264>.
- [24] Antonin Raffin et al. "Stable-Baselines3: Reliable Reinforcement Learning Implementations". In: *Journal of Machine Learning Research* 22.268 (2021), pp. 1–8. URL: <http://jmlr.org/papers/v22/20-1364.html>.
- [25] Adrian Remonda et al. *Formula RL: Deep Reinforcement Learning for Autonomous Racing using Telemetry Data*. 2022. arXiv: 2104.11106 [cs.AI]. URL: <https://arxiv.org/abs/2104.11106>.
- [26] Julian Schrittwieser et al. "Mastering Atari, Go, chess and shogi by planning with a learned model". In: *Nature* 588.7839 (Dec. 2020), pp. 604–609. ISSN: 1476-4687. DOI: 10.1038/s41586-020-03051-4. URL: <http://dx.doi.org/10.1038/s41586-020-03051-4>.
- [27] John Schulman et al. *Proximal Policy Optimization Algorithms*. 2017. arXiv: 1707.06347 [cs.LG]. URL: <https://arxiv.org/abs/1707.06347>.
- [28] Ashish Kumar Shakya, Gopinatha Pillai, and Sohom Chakrabarty. "Reinforcement learning algorithms: A brief survey". In: *Expert Systems with Applications* 231 (2023), p. 120495. ISSN: 0957-4174. DOI: <https://doi.org/10.1016/j.eswa.2023.120495>. URL: <https://www.sciencedirect.com/science/article/pii/S0957417423009971>.
- [29] David Silver et al. *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. 2017. arXiv: 1712.01815 [cs.AI]. URL: <https://arxiv.org/abs/1712.01815>.
- [30] Yunlong Song et al. *Autonomous Overtaking in Gran Turismo Sport Using Curriculum Reinforcement Learning*. 2021. arXiv: 2103.14666 [cs.RO]. URL: <https://arxiv.org/abs/2103.14666>.
- [31] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd. Cambridge, MA: MIT

Press, 2018. URL: <https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf>.

- [32] Mark Towers et al. *Gymnasium: A Standard Interface for Reinforcement Learning Environments*. 2024. arXiv: 2407.17032 [cs.LG]. URL: <https://arxiv.org/abs/2407.17032>.
- [33] Gautham Vasan et al. *Revisiting Sparse Rewards for Goal-Reaching Reinforcement Learning*. 2024. arXiv: 2407.00324 [cs.RO]. URL: <https://arxiv.org/abs/2407.00324>.
- [34] Tingwu Wang et al. *Benchmarking Model-Based Reinforcement Learning*. 2019. arXiv: 1907.02057 [cs.LG]. URL: <https://arxiv.org/abs/1907.02057>.
- [35] Christopher J. C. H. Watkins and Peter Dayan. “Q-learning”. In: *Machine Learning* 8.3 (May 1992), pp. 279–292. ISSN: 1573-0565. DOI: 10.1007/BF00992698. URL: <https://doi.org/10.1007/BF00992698>.
- [36] Ronald J. Williams. “Simple statistical gradient-following algorithms for connectionist reinforcement learning”. In: *Machine Learning* 8.3 (May 1992), pp. 229–256. ISSN: 1573-0565. DOI: 10.1007/BF00992696. URL: <https://doi.org/10.1007/BF00992696>.

IX. APPENDIX

A. State Vector Observations

Name	Dim	α
distnace passed	1	1
speed	1	1000
lateral-dist	1	18
coming-refline-points	$3 * k$	500
wheel-sliding	4	1
surface-materials	4	4
wheel-has-ground-contact	4	1
wheel-damper-absorb	4	1
gearbox	1	5
gear	1	2
rpm	1	12000
freewheeling	1	1

TABLE I
STATE-OBSERVATIONS AND NORMALIZATION FACTOR α

B. Scaled Lateral Distance Reward

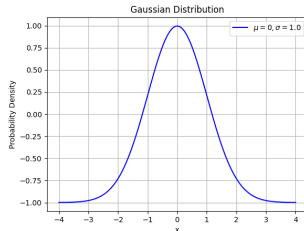


Fig. 13. Scaled Gaussian curve for Lateral Distance. The scaled lateral distance, as given in (3), with $m = 5$, $y = -1$, $s = \frac{1}{3}$.

C. Reward Weights

Reward Terms (r^t)	Reward Weights (λ^t)
Accumulated Distance (r^a)	1
Lateral Distance (r^l)	0.3
Race not Finished (r^f)	-0.05
Velocity (r^v)	0.1
Stuck (r^s)	-20
Race Finished (r^g)	100

TABLE II
REWARDS AND WEIGHTS A

D. Human Racetimes

Map	Map A	Map B
Min Time	31.05	21.01
Max Time	37.81	31.58
Mean Time	34.84	27.57
Std-Deviation	2.03	2.89

TABLE III
RESULTS OF 10 HUMANS PLAYING MAP A AND MAP B. ALL STATS ARE IN SECONDS. TO GET THE NUMBER OF ENVIRONMENT STEPS, MULTIPLY THE RACE TIME IN SECONDS BY 20: $t_s * 20 = n_{env}$.

E. Experiment Performance Figures

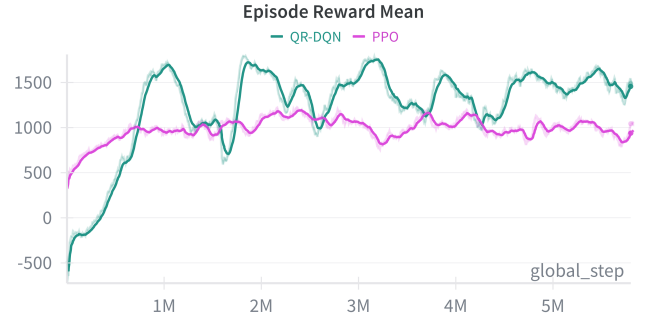


Fig. 14. Episode mean reward of agents trained with PPO (pink) and QR-DQN (green) using the reward function specified in Equation (2).

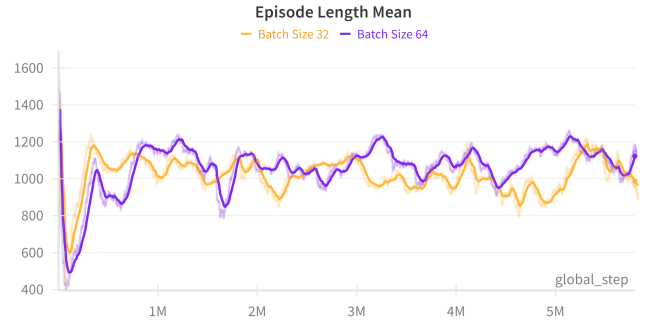


Fig. 15. Comparison of episode length for DQ-DQN with Batch-size 32 (yellow) and Batch-size 64 (purple).

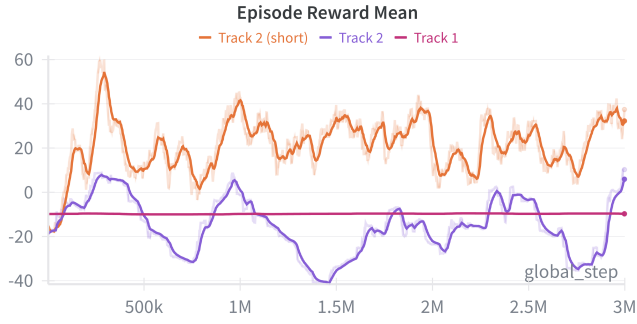


Fig. 16. Mean reward during rollout using a simpler reward function (Experiment C) Scenario 1 (orange), Scenario 2 (purple), Scenario 3 (red).

F. Vision Encoder Architecture

Kernel Size	Out-Channels	Pooling-Size	Activation
5×5	32	2×2	ReLU
5×5	64	2×2	ReLU
3×3	128	2×2	ReLU
3×3	128	No Pooling	ReLU

TABLE IV

DETAILED ARCHITECTURE OF THE LAYERS OF THE VISION ENCODER. ALL OF THE POOLING OPERATIONS ARE MAX-POOLING AND HAVE A STRIDE OF 2.